

Exercise 1: Inventory Management System

Scenario:

You are developing an inventory management system for a warehouse. Efficient data storage and retrieval are crucial.

Steps:

1. Understand the Problem:

- Explain why data structures and algorithms are essential in handling large inventories.
- Discuss the types of data structures suitable for this problem.

2. Setup:

- Create a new project for the inventory management system.

3. Implementation:

- Define a class Product with attributes like **productId**, **productName**, **quantity**, and **price**.
- Choose an appropriate data structure to store the products (e.g., ArrayList, HashMap).
- Implement methods to add, update, and delete products from the inventory.

4. Analysis:

- Analyze the time complexity of each operation (add, update, delete) in your chosen data structure.
- Discuss how you can optimize these operations.

```

import java.util.*;
class Product {
    int productId;
    String productName;
    int quantity;
    double price;
    Product(int productId, String productName, int quantity, double price) {
        this.productId = productId;
        this.productName = productName;
        this.quantity = quantity;
        this.price = price;
    }
    public String toString() {
        return productId + " " + productName + " Qty:" + quantity + " Price:" + price;
    }
}
public class Main {
    public static void main(String[] args) {
        HashMap<Integer, Product> inventory = new HashMap<>();
        inventory.put(101, new Product(101, "Laptop", 10, 50000));
        inventory.put(102, new Product(102, "Mouse", 50, 500));
        System.out.println("Inventory:");
        System.out.println(inventory);
        inventory.put(101, new Product(101, "Laptop", 15, 50000));
        inventory.remove(102);
        System.out.println("\nUpdated Inventory:");
        System.out.println(inventory);
    }
}

```

```

Inventory:
{101=101 Laptop Qty:10 Price:50000.0, 102=102 Mouse Qty:50 Price:500.0}

Updated Inventory:
{101=101 Laptop Qty:15 Price:50000.0}

```

Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. Understand Asymptotic Notation:

- Explain Big O notation and how it helps in analyzing algorithms.
- Describe the best, average, and worst-case scenarios for search operations.

2. Setup:

- Create a class **Product** with attributes for searching, such as **productId**, **productName**, and **category**.

3. Implementation:

- Implement linear search and binary search algorithms.
- Store products in an array for linear search and a sorted array for binary search.

4. Analysis:

- Compare the time complexity of linear and binary search algorithms.
- Discuss which algorithm is more suitable for your platform and why.

```

class Product {
    int productId;
    String productName;
    String category;
    Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }
}

public class Main {
    public static void linearSearch(Product[] products, int id) {
        for (Product p : products) {
            if (p.productId == id) {
                System.out.println("Product Found: " + p.productName);
                return;
            }
        }
        System.out.println("Product Not Found");
    }

    public static void binarySearch(Product[] products, int id) {
        int low = 0, high = products.length - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (products[mid].productId == id) {
                System.out.println("Product Found: " + products[mid].productName);
                return;
            } else if (products[mid].productId < id) {

```

```

                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        System.out.println("Product Not Found");
    }

    public static void main(String[] args) {
        Product[] products = {
            new Product(101, "Laptop", "Electronics"),
            new Product(102, "Mouse", "Electronics"),
            new Product(103, "Keyboard", "Electronics"),
            new Product(104, "Phone", "Electronics")
        };
        System.out.println("Linear Search:");
        linearSearch(products, 103);
        System.out.println("\nBinary Search:");
        binarySearch(products, 103);
    }
}

```

```
Linear Search:
Product Found: Keyboard

Binary Search:
Product Found: Keyboard
```

Exercise 3: Sorting Customer Orders

Scenario:

You are tasked with sorting customer orders by their total price on an e-commerce platform. This helps in prioritizing high-value orders.

Steps:

1. Understand Sorting Algorithms:

- Explain different sorting algorithms (Bubble Sort, Insertion Sort, Quick Sort, Merge Sort).

2. Setup:

- Create a class **Order** with attributes like **orderId**, **customerName**, and **totalPrice**.

3. Implementation:

- Implement **Bubble Sort** to sort orders by **totalPrice**.
- Implement **Quick Sort** to sort orders by **totalPrice**.

4. Analysis:

- Compare the performance (time complexity) of Bubble Sort and Quick Sort.

Discuss why Quick Sort is generally preferred over Bubble Sort.

```

class Order {
    int orderId;
    String customerName;
    double totalPrice;
    Order(int orderId, String customerName, double totalPrice) {
        this.orderId = orderId;
        this.customerName = customerName;
        this.totalPrice = totalPrice;
    }
}

public class Main {
    static void display(Order[] orders) {
        for (Order o : orders) {
            System.out.println(o.orderId + " " + o.customerName + " Rs." + o.totalPrice);
        }
    }

    static void bubbleSort(Order[] orders) {
        int n = orders.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (orders[j].totalPrice > orders[j + 1].totalPrice) {
                    Order temp = orders[j];
                    orders[j] = orders[j + 1];
                    orders[j + 1] = temp;
                }
            }
        }
    }
}

```

```

    }
}

static void quickSort(Order[] orders, int low, int high) {
    if (low < high) {
        int p = partition(orders, low, high);
        quickSort(orders, low, p - 1);
        quickSort(orders, p + 1, high);
    }
}

static int partition(Order[] orders, int low, int high) {
    double pivot = orders[high].totalPrice;
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (orders[j].totalPrice < pivot) {
            i++;
            Order temp = orders[i];
            orders[i] = orders[j];
            orders[j] = temp;
        }
    }
    Order temp = orders[i + 1];
    orders[i + 1] = orders[high];
    orders[high] = temp;
    return i + 1;
}

```

```

    public static void main(String[] args) {
        Order[] orders = {
            new Order(101, "Amit", 5000),
            new Order(102, "Riya", 2000),
            new Order(103, "John", 8000),
            new Order(104, "Sara", 3000)
        };
        System.out.println("Bubble Sort:");
        bubbleSort(orders);
        display(orders);
        System.out.println("\nQuick Sort:");
        quickSort(orders, 0, orders.length - 1);
        display(orders);
    }
}

```

```

Bubble Sort:
102 Riya Rs.2000.0
104 Sara Rs.3000.0
101 Amit Rs.5000.0
103 John Rs.8000.0

Quick Sort:
102 Riya Rs.2000.0
104 Sara Rs.3000.0
101 Amit Rs.5000.0
103 John Rs.8000.0

```

Exercise 4: Employee

Management System

Scenario:

You are developing an employee management system for a company. Efficiently managing employee records is crucial.

Steps:

1. Understand Array Representation:

- Explain how arrays are represented in memory and their advantages.

2. Setup:

- Create a class Employee with attributes like **employeeid**, **name**, **position**, and **salary**.

3. Implementation:

- Use an array to store employee records.
- Implement methods to **add**, **search**, **traverse**, and **delete** employees in the array.

4. Analysis:

- Analyze the time complexity of each operation (add, search, traverse, delete).
- Discuss the limitations of arrays and when to use them.

```

static void displayEmployees() {
    System.out.println("Employee List:");
    for (int i = 0; i < count; i++) {
        System.out.println(employees[i].employeeId + " "
            + employees[i].name + " "
            + employees[i].position + " Rs."
            + employees[i].salary);
    }
}

static void deleteEmployee(int id) {
    for (int i = 0; i < count; i++) {
        if (employees[i].employeeId == id) {
            for (int j = i; j < count - 1; j++) {
                employees[j] = employees[j + 1];
            }
            employees[count - 1] = null;
            count--;
            System.out.println("Employee Deleted");
            return;
        }
    }
}

public static void main(String[] args) {
    addEmployee(new Employee(101, "Shreyasi", "Manager", 50000));
    addEmployee(new Employee(102, "Rahul", "Developer", 40000));
    addEmployee(new Employee(103, "Priya", "Tester", 35000));

    displayEmployees();
    searchEmployee(102);
    deleteEmployee(102);
    displayEmployees();
}
}

```

```

Employee List:
101 Shreyasi Manager Rs.50000.0
102 Rahul Developer Rs.40000.0
103 Priya Tester Rs.35000.0
Found: Rahul
Employee Deleted
Employee List:
101 Shreyasi Manager Rs.50000.0
103 Priya Tester Rs.35000.0

```

Exercise 5: Task Management System

Scenario:

You are developing a task management system where tasks need to be added, deleted, and traversed efficiently.

Steps:

1. Understand Linked Lists:

- Explain the different types of linked lists (Singly Linked List, Doubly Linked List).

2. Setup:

- Create a class **Task** with attributes like **taskId**, **taskName**, and **status**.

3. Implementation:

- Implement a singly linked list to manage tasks.
- Implement methods to **add**, **search**, **traverse**, and **delete** tasks in the linked list.

4. Analysis:

- Analyze the time complexity of each operation.
- Discuss the advantages of linked lists over arrays for dynamic data.

```

class Task {
    int taskId;
    String taskName;
    String status;
    Task next;
    Task(int taskId, String taskName, String status) {
        this.taskId = taskId;
        this.taskName = taskName;
        this.status = status;
        this.next = null;
    }
}

public class Main {
    static Task head = null;
    static void addTask(int id, String name, String status) {
        Task newTask = new Task(id, name, status);
        if (head == null) {
            head = newTask;
        } else {
            Task temp = head;
            while (temp.next != null) {
                temp = temp.next;
            }
            temp.next = newTask;
        }
    }
}

```

```

    static void searchTask(int id) {
        Task temp = head;
        while (temp != null) {
            if (temp.taskId == id) {
                System.out.println("Task Found: " + temp.taskName);
                return;
            }
            temp = temp.next;
        }
        System.out.println("Task Not Found");
    }

    static void displayTasks() {
        Task temp = head;
        while (temp != null) {
            System.out.println(temp.taskId + " " + temp.taskName + " " + temp.status);
            temp = temp.next;
        }
    }

    static void deleteTask(int id) {
        if (head == null)
            return;
        if (head.taskId == id) {
            head = head.next;
            return;
        }
    }
}

```

```

    }
    Task temp = head;
    while (temp.next != null && temp.next.taskId != id) {
        temp = temp.next;
    }
    if (temp.next != null) {
        temp.next = temp.next.next;
        System.out.println("Task Deleted");
    }
}

public static void main(String[] args) {
    addTask(1, "Coding", "Pending");
    addTask(2, "Testing", "Completed");
    addTask(3, "Documentation", "Pending");
    System.out.println("Task List:");
    displayTasks();
    searchTask(2);
    deleteTask(2);
    System.out.println("\nUpdated Task List:");
    displayTasks();
}
}

```

```

Task List:
1 Coding Pending
2 Testing Completed
3 Documentation Pending
Task Found: Testing
Task Deleted

Updated Task List:
1 Coding Pending
3 Documentation Pending

```

Exercise 6: Library Management System

Scenario:

You are developing a library management system where users can search for books by title or author.

Steps:

1. Understand Search Algorithms:

- Explain linear search and binary search algorithms.

2. Setup:

- Create a class **Book** with attributes like **bookId**, **title**, and **author**.

3. Implementation:

- Implement linear search to find books by title.
- Implement binary search to find books by title (assuming the list is sorted).

4. Analysis:

- Compare the time complexity of linear and binary search.
- Discuss when to use each algorithm based on the data set size and order.

```

class Book {
    int id;
    String title;

    Book(int id, String title) {
        this.id = id;
        this.title = title;
    }
}

public class Main {

    static void linearSearch(Book[] books, String title) {
        for (Book b : books) {
            if (b.title.equals(title)) {
                System.out.println("Book Found: " + b.title);
                return;
            }
        }
        System.out.println("Book Not Found");
    }

    static void binarySearch(Book[] books, String title) {
        int low = 0, high = books.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;

            if (books[mid].title.equals(title)) {
                System.out.println("Book Found: " + books[mid].title);
                return;
            } else if (books[mid].title.compareTo(title) < 0) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        System.out.println("Book Not Found");
    }

    public static void main(String[] args) {

        Book[] books = {
            new Book(1, "C"),
            new Book(2, "Java"),
            new Book(3, "Python")
        };

        linearSearch(books, "Java");
        binarySearch(books, "Java");
    }
}

```

```
Book Found: Java
Book Found: Java
```

Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Steps:

1. Understand Recursive Algorithms:

- Explain the concept of recursion and how it can simplify certain problems.

2. Setup:

- Create a method to calculate the future value using a recursive approach.

3. Implementation:

- Implement a recursive algorithm to predict future values based on past growth rates.

4. Analysis:

- Discuss the time complexity of your recursive algorithm.
- Explain how to optimize the recursive solution to avoid excessive computation.

```
public class Main {
    static double futureValue(double amount, double rate, int years) {
        if (years == 0)
            return amount;
        return futureValue(amount * (1 + rate), rate, years - 1);
    }
    public static void main(String[] args) {
        double amount = 1000;
        double rate = 0.10;
        int years = 3;
        double result = futureValue(amount, rate, years);
        System.out.println("Future Value = " + result);
    }
}
```

```
Future Value = 1331.0
```